Week-5 | Session-1

Week-5 to week-12

Supervised learning

- **Regression**
- Classification

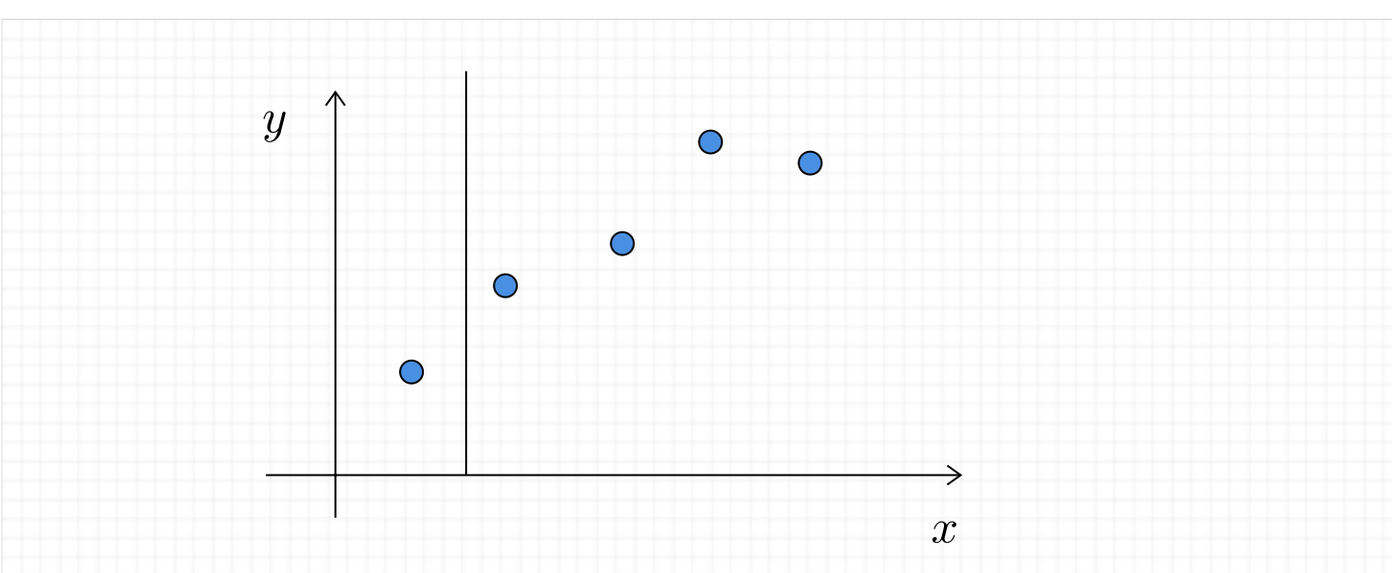
<u>Supervised learning</u>

$$X \rightarrow d \times n$$

$$X, y$$

For example:

$$x \rightarrow \text{calories consumed}$$
$$y \rightarrow \text{how many hours you are able to play}$$



Goal: Use the (training) data given to you and come up with a mapping (function) from features to labels that will help you predict the label on (test) data that you have not seen so far

- **learning** the function (learning from data)

- prediction on unseen data (**generalization**)

When the label $y \in \mathbb{R}$, we call the problem regression.
When the label $y \in \{0, 1, \cdots, k\}$, we call the problem classification.

$$x \rightarrow \text{calories consumed}$$
$$y \rightarrow \text{is your energy level high or low}$$

Binary classification problem

$$y = \{0, 1\}$$

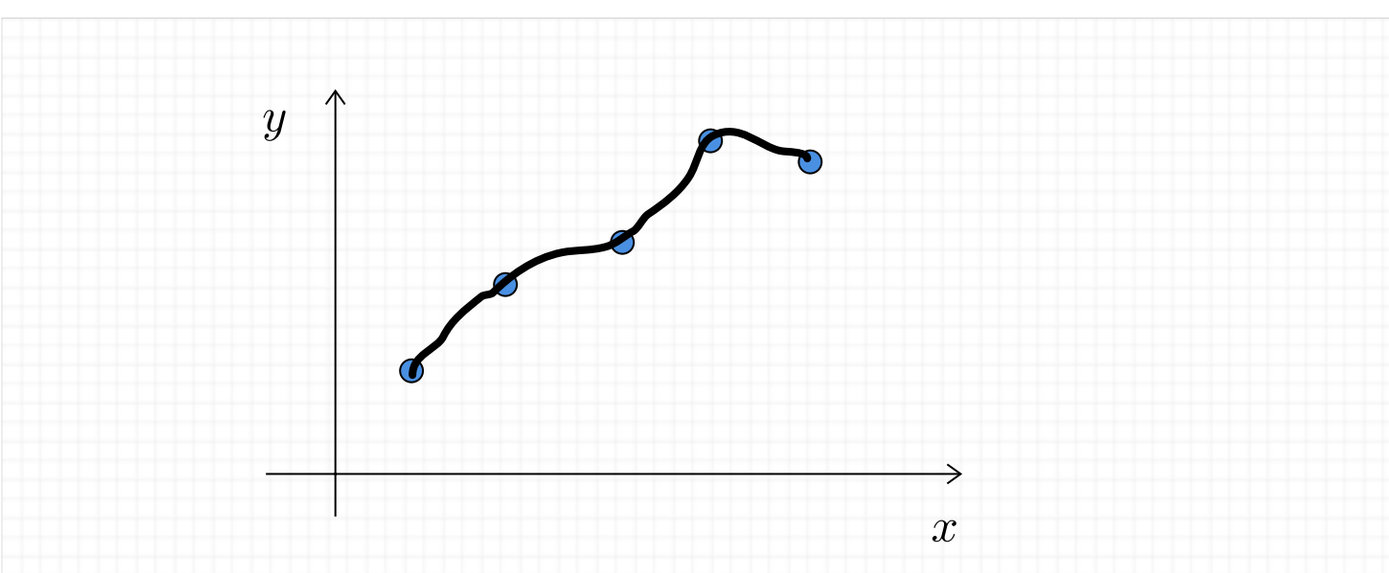
<u>Regression</u>

$D = \{(x_1, y_1), \cdots, (x_n, y_n)\}$

$$x_i \in \mathbb{R}^d$$
$$y_i \in \mathbb{R}$$

Learn a function from $f : \mathbb{R}^d \rightarrow \mathbb{R}$:

$$f(x_i) = y_i$$



Data:

- signal (pattern)
- noise
  - device used for measurement could be noisy

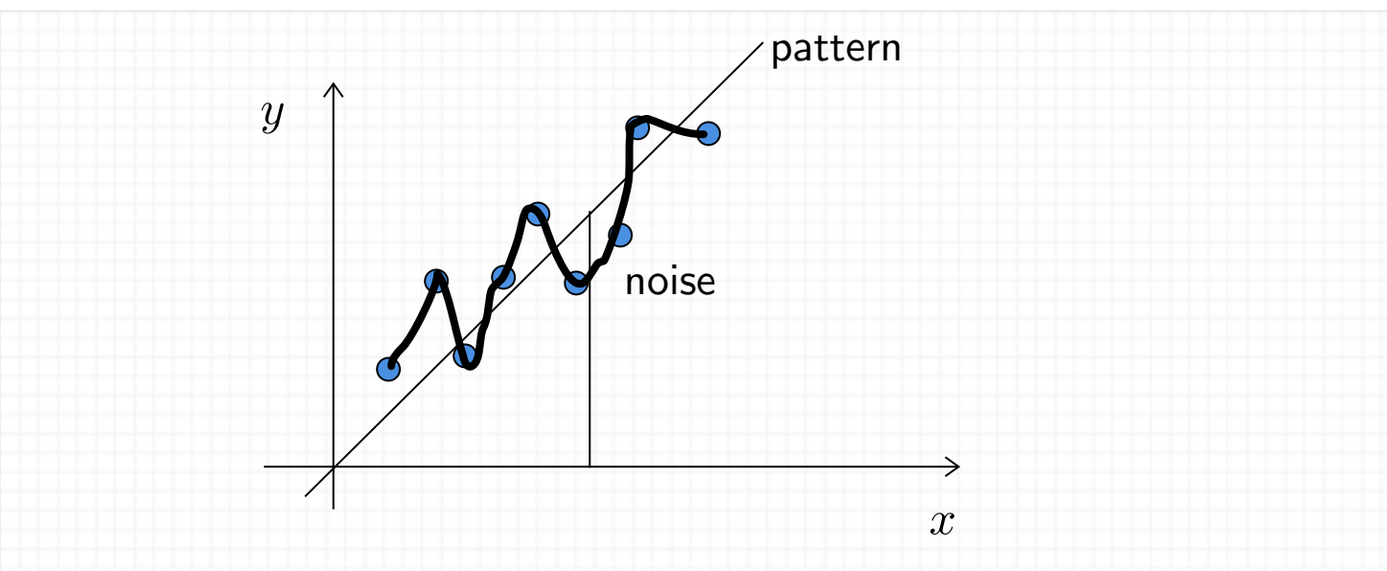
For this dataset, what is the pattern:

- linear
  - as calories increases (within a range), the hours of play increases

$$f(x_i) = y_i$$

- You learn not just the pattern, but the noise as well

Data that is given to you is called training data. Whatever data you use to learn the function is "training data".



Overfitting: performing well in training data, but performing poorly in the test data
Fit a function or a curve to a dataset
Glorified Curve Fitting

How do you measure performance?

$$f(x_i) \rightarrow \text{predicted label}$$
$$y_i \rightarrow \text{true label}$$

Error in prediction:

$$e_i = f(x_i) - y_i$$

<u>Performance metrics</u>

Sum of squared errors

$$SSE = \sum_{i=1}^{n} [f(x_i) - y_i]^2$$

Means squared error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^{n} [f(x_i) - y_i]^2$$

Per-data-point error (units squared hours of play)

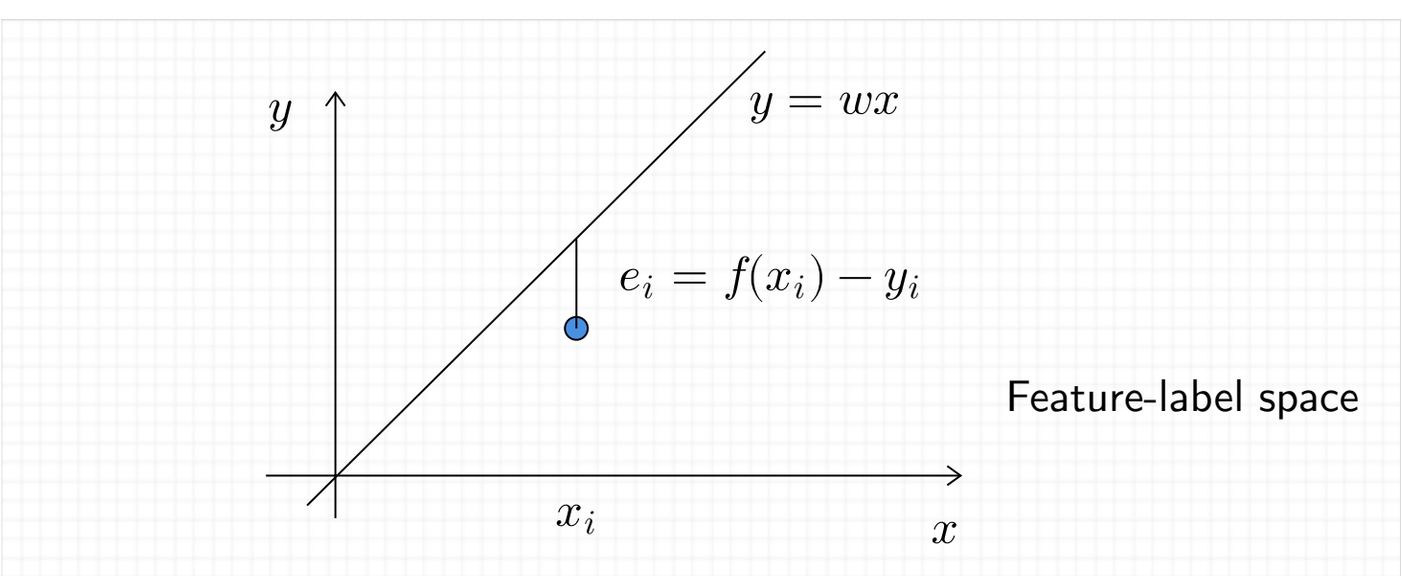
RMSE:

$$RMSE = \sqrt{\frac{1}{n} \cdot \sum_{i=1}^{n} [f(x_i) - y_i]^2}$$

Per-data-point error (units hours of play)

<u>Linear regression</u>

$$f(x) = wx$$



Loss function (error function)

$$L(w) = \sum_{i=1}^{n} [wx_i - y_i]^2$$

How much loss do you incur by predicting the label for x_i as wx_i instead of the true label y_i. Find the line which will give you the least loss (error).

$$\min_{w} \; L(w)$$

Best-fit line

$$\min_{w} \sum_{i=1}^{n} [wx_i - y_i]^2$$

Unconstrained

$$\frac{dL}{dw} = \sum_{i=1}^{n} 2(wx_i - y_i)x_i = 0$$

$$\sum_{i=1}^{n} \left[wx_i^2 - x_i y_i\right] = 0 \Longrightarrow w\left(\sum_{i=1}^{n} x_i^2\right) - \left(\sum_{i=1}^{n} x_i y_i\right) = 0$$

Simple linear regression

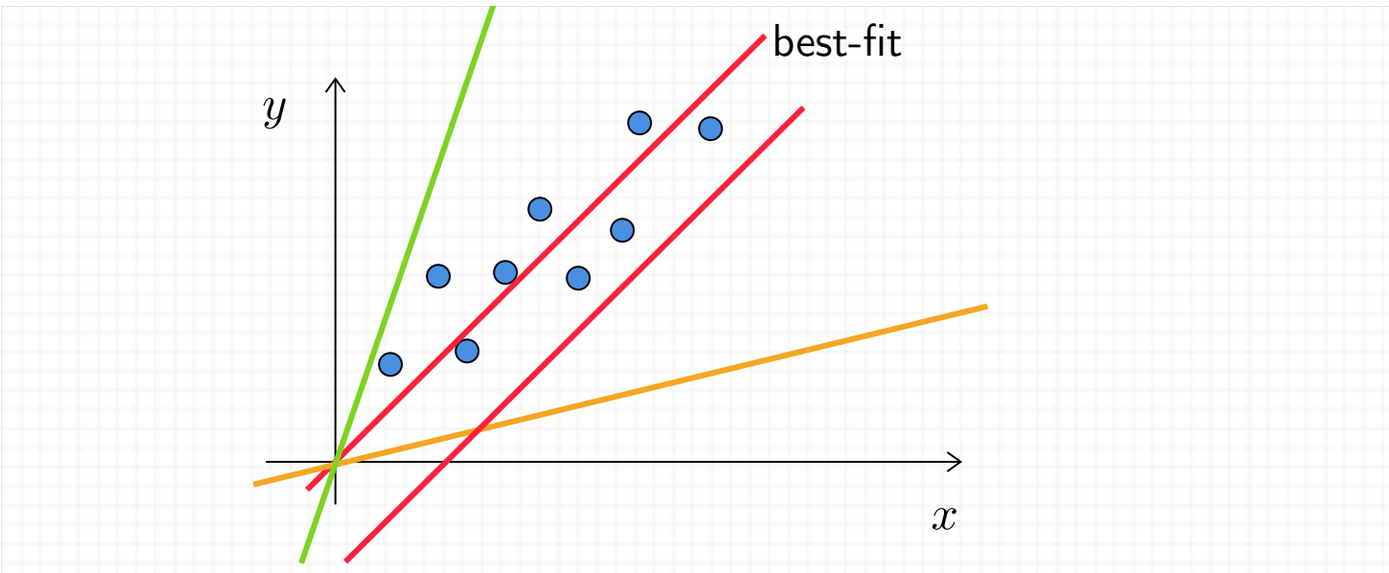
One feature

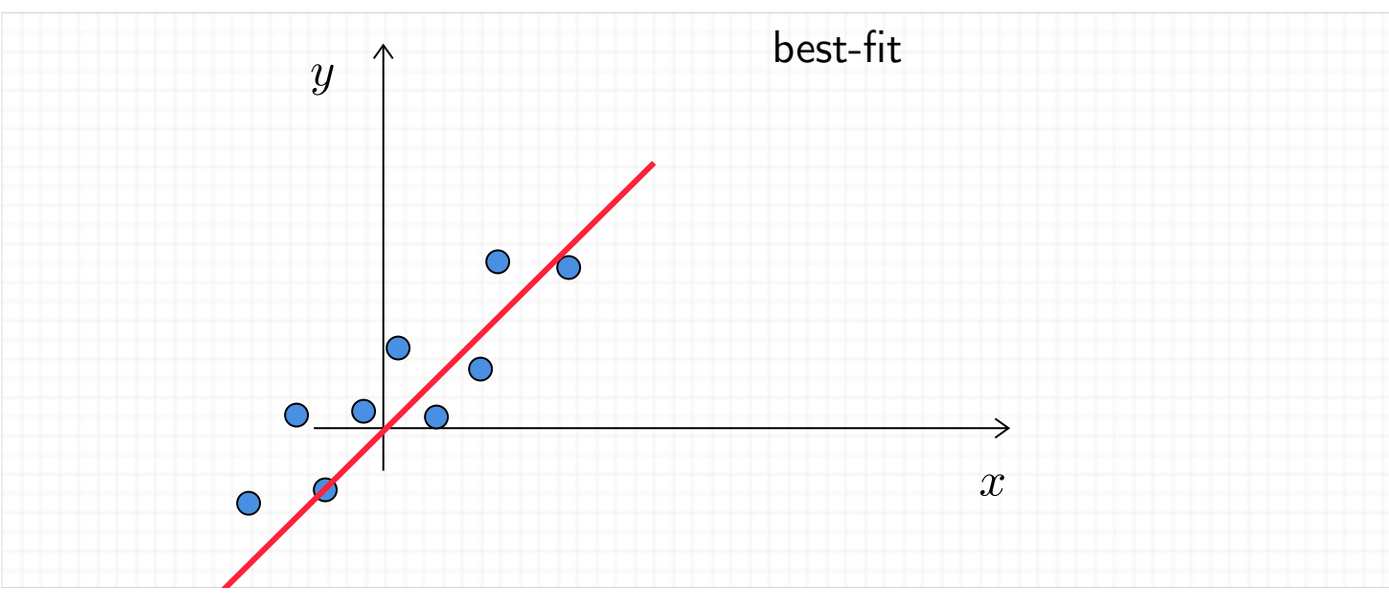
$$\widehat{w} = \frac{\sum_{i=1}^{n} x_i y_i}{\sum_{i=1}^{n} x_i^2}$$

We are searching in the space of lines:

Best-fit line depends two things:

- features
- feature-label interaction





If the data is centered:

- $\sum x_i = 0$
- $\sum y_i = 0$

Then,

$$f(x) = wx + b$$

$$\widehat{b} = 0$$

<u>Multiple features</u>

ISLR $\rightarrow$ Introduction to Statistical Learning using R

ISLP $\rightarrow$ Introduction to Statistical Learning using Python

Advertising data
Launched a new product:

- Newspapers
  - x_1
- Television
  - x_2
- Google
  - x_3

You have to spend some money to advertise your product in each of these channels.

Sales is a function of the advertising budget.

$$y = f(x_1, x_2, x_3)$$

Historical data

- 2023
  - 12 months of sales data
- $(x_{11}, x_{12}, x_{13}, y_1), (x_{21}, x_{22}, x_{23}, y_2), \ \cdots$

Fit a model that will predict the sales given the advertising budget.

$$x \in \mathbb{R}^3$$
$$y \in \mathbb{R}$$

$$f : \mathbb{R}^3 \rightarrow \mathbb{R}$$

$$L(f) = \sum [f(x_i) - y_i]^2$$

$$\min_f \ L(f)$$

<u>Linear regression</u>

$$f(x) = w_1 x_1 + w_2 x_2 + w_3 x_3$$

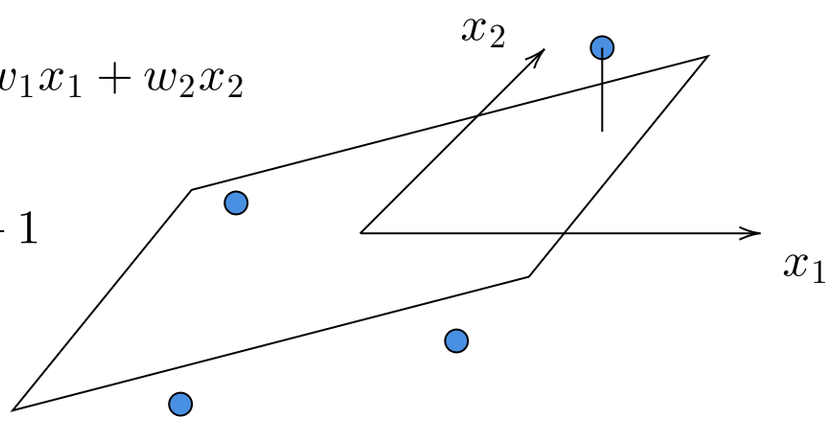
Assumption: The sales depends linearly on the advertising budget

$$f(x) = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = w^T x$$

best-fit plane

$y = w_1 x_1 + w_2 x_2$

$d + 1$



best-fit hyperplane

SSE

$$w \in \mathbb{R}^d$$

$$L(w) = \sum_{i=1}^{n} \left(w^T x_i - y_i \right)^2$$

$$\nabla L(w) = \begin{bmatrix} \dfrac{\partial L}{\partial w_1} \\ \vdots \\ \dfrac{\partial L}{\partial w_d} \end{bmatrix}$$

$$L(w) = \sum_{i=1}^{n} \left(w^T x_i - y_i\right)^2$$

$$\begin{aligned}
\nabla L(w) &= \sum_{i=1}^{n} 2\left(\mathbf{w}^T \mathbf{x}_i - y_i\right)\mathbf{x}_i \\
&= 2\sum_{i=1}^{n} x_i\left(w^T x_i - y_i\right) \\
&= 2\sum_{i=1}^{n} x_i\left(x_i^T w - y_i\right) \\
&= 2\sum_{i=1}^{n} x_i x_i^T w - x_i y_i \\
&= 2\left(\sum x_i x_i^T\right) w - 2\sum x_i y_i \\
&= 2\left[\mathbf{X}\mathbf{X}^T\mathbf{w} - \mathbf{X}\mathbf{y}\right]
\end{aligned}$$

$$Xy = \begin{bmatrix} | & & | \\ x_1 & \cdots & x_n \\ | & & | \end{bmatrix} \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = y_1 x_1 + \cdots + y_n x_n$$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$$L(\mathbf{w}) = \sum_{i=1}^{n} \left(\mathbf{w}^T \mathbf{x}_i - y_i\right)^2$$

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = 2\left[\mathbf{X}\mathbf{X}^T\mathbf{w} - \mathbf{X}\mathbf{y}\right]$$

For minimizing $L(\mathbf{w})$, you set the gradient to the zero vector.

$$\nabla L(\mathbf{w}) = \mathbf{0} \Longrightarrow \mathbf{X}\mathbf{X}^T\mathbf{w} = \mathbf{X}\mathbf{y}$$

This is a system of equations:

$$XX^T \to d \times d$$

- d equations in d unknowns
- This is system is always consistent. It will always have at least one solution.

$L(\mathbf{w})$ is a convex function. All local minima are global minima.

<u>Normal equations</u>

$$\mathbf{X}\mathbf{X}^T\mathbf{w} = \mathbf{X}\mathbf{y}$$

$$\left(\mathbf{X}\mathbf{X}^T\right)^{\dagger}(\mathbf{X}\mathbf{y})$$

$$\left(\mathbf{X}\mathbf{X}^T\right)^{-1}(\mathbf{X}\mathbf{y})$$

Least squares problem: This has existed from the days of Gauss. This problem is closely tied to astronomy. Predict the path of celestial bodies from observations.

Pseudo-inverse:

Moore-Penrose inverse exists for all matrices.

SVD of a matrix to compute its pseudo-inverse.

You can find the pseudo-inverse

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix}$$

NumPy

np.linalg.pinv(X) $\rightarrow$

$$\mathbf{w} = \left(\mathbf{X}\mathbf{X}^T\right)^{-1}\mathbf{X}\mathbf{y}$$

- XX^T $\rightarrow$ feature dependence
- Xy $\rightarrow$ feature-label interaction

The final weight vector depends on both features and labels

$$y = w_1x_1 + w_2x_2 + w_3x_3$$

$w^* = [1, 2, 5]$ is the weight vector learnt from the training data

$$y = -x_1 + 2x_2 + 5x_3$$

- Google is five times more effective in driving sales compared to newspapers
- TV is two more times as effective as newspapers

The weights tell you how important each feature is.

Which feature is more valuable in predicting the outcome than the others.

<u>Geometric perspective</u>

- w

$$f(x) = w^T x$$

Dataset n points
Predicted vector

$$\mathbf{y} \rightarrow \text{True label vector}, \;\; y \in \mathbb{R}^n$$

$$\widehat{\mathbf{y}} = \begin{bmatrix} x_1^T w \\ \vdots \\ x_n^T w \end{bmatrix} = \begin{bmatrix} - & \mathbf{x}_1^T & - \\ & \vdots & \\ - & \mathbf{x}_n^T & - \end{bmatrix} \mathbf{w} = \mathbf{X}^T \mathbf{w}$$

$$L(\mathbf{w}) = ||\widehat{\mathbf{y}} - \mathbf{y}||^2 = ||\mathbf{X}^T \mathbf{w} - \mathbf{y}||^2$$

$$\mathbf{X}^T \mathbf{w} \approx \mathbf{y}$$

We are trying to find a w which will make $X^T w$ as close as possible to y. The vector closest to y is therefore the projection of y onto the columns of X^T.

$$X^T w$$

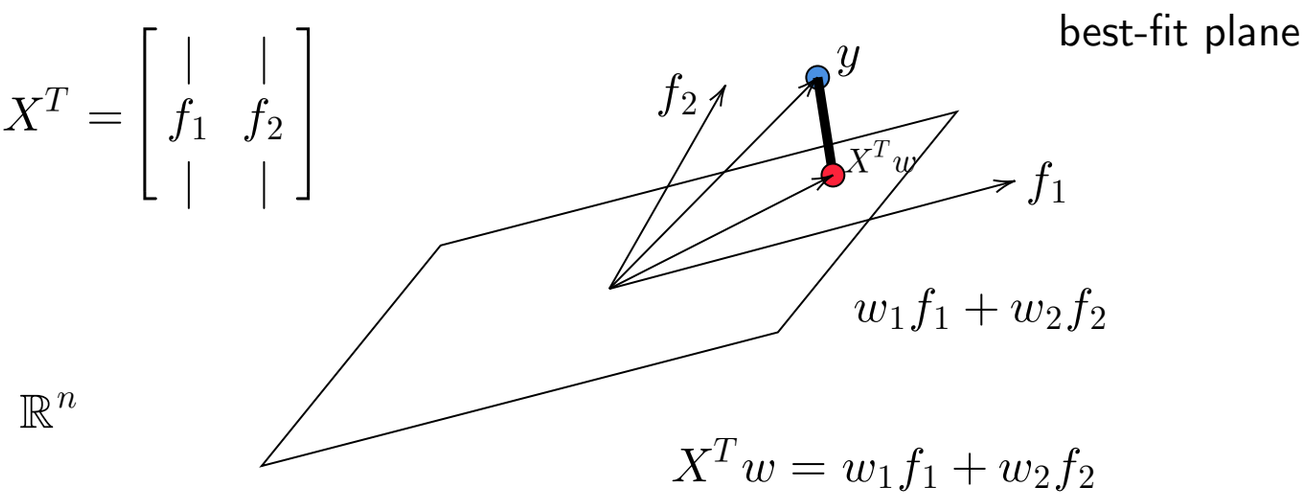
It is a linear combination of the columns of X^T. You are trying to find a vector in the span of the columns of X^T that is as close as possible to y.

$$||\mathbf{X}^T\mathbf{w} - \mathbf{y}||^2$$

$$X^T \to n \times d$$

Each column will correspond to one feature (but that feature value for all the data-points)

$$X^T = \begin{bmatrix} | & | \\ f_1 & f_2 \\ | & | \end{bmatrix}$$



The projection of y onto the plane spanned by the columns of X^T is the one closest to y.

$$\left(y - X^T w\right) \perp \text{columns of } X^T$$

$$\left(y - X^T w\right) \perp \text{rows of } X$$

$$\mathbf{X}\left(\mathbf{y} - \mathbf{X}^T\mathbf{w}\right) = \mathbf{0}$$

$$\mathbf{X}\mathbf{X}^T\mathbf{w} = \mathbf{X}\mathbf{y}$$

Projecting the label vector on to the space spanned by the features.

Learning from data

$$\mathbb{R}^{d+1} \rightarrow \text{feature-label space}$$

Loss function: the distance between the data-point and the prediction on the hyperplane (parallel to the label axis (y-axis))